ENGINEERING FIELD GUIDE

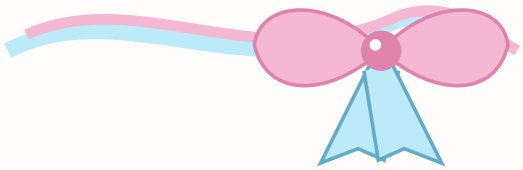
Ribbon Protocol

Guardrails for a multi-LLM engineering belt

Plan with intelligence. Implement with constraints. Verify with evidence.

Planner + Coding Agent + Trusted Harness

Practical recipes, stop-the-line gates, adoption phases, metrics, and model roles.



A Sylveon-inspired reference handbook

October 2026 | Working edition 1.0

Sukoshi zutsu, kakujitsu ni - little by little, with certainty.

Prepared for deep-read & reread

START HERE

Your operating system in one page

Five connected practices that make mixed-capability agents dependable.

The core operating loop

01
ARCHITECT
Resolve semantics and risk

02
CONTRACT
Encode allowed choices

03
EXECUTE
Write bounded changes

04
VERIFY
Run trusted acceptance

THE CONTRACT IS THE BOUNDARY

The planner owns correctness decisions; the coder owns bounded implementation choices; the harness owns independent acceptance. **Do not outsource the final definition of success to the agent writing the patch.**

What is inside this guide

01 **Key values & do/don't rules**
Choose by accepted outcomes, not impressions.

02 **Guardrails and execution recipe**
Preflight, invariants, boundaries, stop line.

03 **Adoption phases**
Start small; automate only after evidence.

04 **Feedback extraction**
Turn every failure into a tracked improvement.

05 **Model utilities**
Route coder, fixer, planner, and architect roles.

READING NOTE

Model names in the role matrix are **candidate assignments from our discussion**, not universal quality rankings. The contract and evaluation method should survive provider or model changes.

01 / KEY VALUES

What to optimize when using many LLMs

Use system-level outcomes - not tokens, leaderboard positions, or one lucky patch.

Engineering values & observable signals

VALUE	WHAT IT REALLY MEANS	HOW TO OBSERVE IT
Correctness	Accepted behavior and invariant checks	Independent tests; edge cases; real expected outcomes
Bounded autonomy	Decisions delegated intentionally	Scope, decision ownership, and escalation points
Evidence	Proof, not self-reported success	Actual command outputs, diff, test logs
Safety & data hygiene	Blast radius and disclosure risk	Permissions, secret redaction, approved providers
Repairability	Time to recover from a failed first pass	Failure classification; repair effort; clean rollback
Independence	Reviewer not grading its own work	Protected acceptance tests; independent reviewer
Cost per accepted result	Total spend and effort per merged change	Subscriptions, tokens, retries, review, latency
Reproducibility	Comparable runs over time	Same repo snapshot, tools, test harness and task class
Context quality	Facts precede guesses	Exact symbols, paths, references and verified baseline

THE CRUZEIRO DO SUL (SOUTHERN CROSS) METRIC

Accepted, correct changes per unit of total engineering effort. A cheap model is not economical if it increases senior-engineer review, retries, or defect risk.

01 / KEY VALUES

The practical do / don't list

Tiny operational decisions often dominate the economics of your model belt.

Operating principles you can audit

DIMENSION	DO / GOOD PRACTICE	DON'T / ANTI-PATTERN
Model routing	Choose from measured task classes and risk.	Route every task to the cheapest model by default.
Task design	Specify expected outcomes and forbidden scope.	Write elaborate prompts without testable results.
Planning	Resolve invariants and interfaces before delegation.	Ask a lightweight coder to invent transaction semantics.
Execution	Give bounded local choices and a repair budget.	Let repeated retries turn into unbounded exploration.
Validation	Verify in a trusted runner outside agent control.	Accept a statement that tests passed as proof.
Escalation	Escalate ambiguity or semantic failure early.	Treat every red test as a syntax-only issue.
Experiments	Compare on the same tasks with recorded effort.	Compare anecdotes across different repositories.
Security	Use approved providers and sandbox credentials.	Copy proprietary code or secrets into personal agents.

A USEFUL PRE-FLIGHT QUESTION

If the coder has to invent a domain policy, design a new abstraction, or decide the correctness criteria, **the task is still a planning task**. Send it back to the architect.

02 / PLANNER GUARDRAILS

Anatomy of a genuinely executable recipe

Write contracts that narrow the reasoning burden without scripting every line.

Eight parts of a good handoff

01 / Goal & non-goals

Observable behavior plus what must remain unchanged.

02 / Verified context

Existing files, signatures, patterns, baseline state.

03 / Invariants

Properties that must be true across normal and failure paths.

04 / Decisions owned

Architecture set by planner; local choices left to coder.

05 / Bounded scope

Allowed files, protected files, dependency restrictions.

06 / Acceptance oracle

Behavior matrix + tests independent from executor.

07 / Stop conditions

Ambiguity, forbidden edits, missing preconditions, failed retries.

08 / Completion evidence

Patch summary, diffs, commands, test results, known gaps.

THE DECISION-BUDGET RULE

The planner decides **semantics, failure modes, public interfaces, invariants, and compatibility**. The coder decides **local naming, formatting, minor decomposition, and implementation mechanics** within those bounds.

02 / PLANNER GUARDRAILS

A seven-step, stop-the-line execution protocol

A coder is allowed to report BLOCKED. An invented answer is not progress.

The state machine

1. **PLAN**
   Premium model resolves design and writes acceptance cases.
2. **PREFLIGHT**
   Coder verifies files, APIs, patterns, baseline and prerequisites - read only.
3. **AUTHORIZE**
   Harness checks scope and readiness; proceed only when all gates are ready.
4. **IMPLEMENT**
   Coder makes the smallest change within the allowlist.
5. **VERIFY**
   Trusted runner checks targeted tests, hidden cases and diff policy.
6. **REPAIR / ESCALATE**
   Fix mechanical errors within budget; escalate semantic uncertainty.
7. **HANDOFF**
   Return patch, evidence, risks, and status PASS / BLOCKED / ESCALATED.

STOP THE LINE: HARD GATE

The agent must stop when a prerequisite is false, the task requires a protected edit, the API is missing, acceptance behavior is unresolved, or a semantic failure persists. **Only the planner or owner may revise the contract.**

Preflight output: `READY` with verified evidence and expected file list, or `BLOCKED` with concrete missing facts. The coder must not silently "fill in" preconditions.

02 / PLANNER GUARDRAILS

A reusable planner-to-coder contract

Illustrative Clojure event-processing task: bounded behavior, explicit assumptions.

implementation-contract.yaml

```yaml
schema_version: 1
task_id: payment-consumer-014
objective: "Deduplicate delivery of a payment event"
out_of_scope: ["schema", "dependencies", "API redesign"]
context:
  read_first: [src/payment/consumer.clj,
               src/payment/repository.clj,
               test/payment/consumer_test.clj]
  reference_pattern: src/order/consumer.clj

preconditions:
  - "A durable unique event-ID constraint exists"
  - "Dedup + transition share one DB transaction"
  - "No irreversible external effects in this operation"
behavior:
  - {given: "first event A", expect: "one committed transition"}
  - {given: "duplicate A", expect: "no additional transition"}
  - {given: "failure then retry", expect: "no partial DB state"}
  - {given: "concurrent A", expect: "at most one transition"}
invariants:
  - "A repeated event ID cannot commit a second transition"
  - "Normal non-duplicate behavior remains unchanged"

scope:
  writable: [src/payment/consumer.clj,
             test/payment/consumer_test.clj]
  protected: ["resources/migrations/**", "docs/adr/**"]
  forbidden: ["new dependency", "public API change"]
verification:
  baseline: "./scripts/test-payment-consumer.sh"
  acceptance: "trusted harness executes protected tests"
  diff_policy: "changed paths must be in scope.writable"

stop_line:
  - "precondition cannot be verified"
  - "correctness requires edits outside writable scope"
  - "transaction semantics are ambiguous"
  - "two focused repairs failed"
completion: ["changed paths", "actual test results",
             "diff summary", "risks", "PASS|BLOCKED"]
```

Important: this contract is valid only if its preconditions can be verified. The DB guarantee does not imply exactly-once external effects. The shown test command is a repository-specific placeholder to replace before use.

02 / PLANNER GUARDRAILS

Enforce boundaries - do not merely request them

When correctness matters, critical constraints belong in tools and policy.

From a written rule to a real control

CONSTRAINT	DETECTOR	ON VIOLATION	RESUME WHEN
Protected files	Filesystem / sandbox	Block write immediately	Owner approves scope change
Dependency drift	Manifest + lockfile diff	Reject patch	Dependency decision documented
Public API change	Contract tests / diff policy	Reject and escalate	Architect revises interface contract
Tests fail	Trusted test runner	Focused repair within budget	Passing evidence
Wrong business meaning	Independent acceptance	Stop semantic repair loop	Planner resolves ambiguity
Secrets / egress	Sandbox, credentials, network policy	Deny access / execution	Approved environment only
Missing precondition	Read-only preflight gate	BLOCKED; no edits	Evidence or new plan
Repeated failure	Attempt counter + failure class	Escalate model / replan	New contract or owner decision

Where enforcement belongs

Planner: goal, decisions, invariants, exceptions. **Coder:** minimal patch, local verification, truthful evidence. **Harness:** write policies, protected acceptance, secret boundaries, time/attempt caps, CI. **Human:** risk acceptance and permissions.

THE TRUST BOUNDARY

Run trusted acceptance checks from an environment the coder cannot edit. A protected path inside a fully editable workspace is weaker than an independent CI checkout or sandbox with separate permissions.

03 / ADOPTION

Adopt in phases, using evidence-based gates

The clock is a planning aid; measured conditions decide when to advance.

PHASE 0 — Today / next task

Contract + scope + required evidence

Every delegated task has a clear outcome, path allowlist, stop rules and completion report.

Exit gate: Start immediately. No automation prerequisite.

PHASE 1 — Within 1-2 weeks

Read-only preflight + retry budget

Require API/path verification, baseline tests and one bounded implementation + up to two focused repairs.

Exit gate: Advance after ~10 real tasks with captured failure reasons.

PHASE 2 — After 15-30 tasks

Trusted, executable acceptance

Put file-scope, dependencies, tests and forbidden changes in CI or the sandbox. Protect independent acceptance cases.

Exit gate: Advance when you see repeatable failures or policy violations.

PHASE 3 — After 30-50 tasks

Failure taxonomy + model routing

Track effective cost, acceptance rate, retries, review time, and premium escalation per task class.

Exit gate: Advance when the same failure type recurs >=3 times, or routing economics become visible.

PHASE 4 — After stable results / ~1-3 months

Automated contracts and orchestration

Generate contracts, evaluate task risk, route models, capture telemetry, create clean worktrees.

Exit gate: Only if orchestration saves more time than it costs to maintain.

Anti-pattern: building a multi-agent orchestration platform before you can explain what the first 20 failures were. Start with a readable contract and a single trusted verification command.

04 / FEEDBACK

Measure the outcome, not the cleverness

A small ledger is enough to prove whether better contracts beat stronger models.

Your minimum scorecard

METRIC	DEFINITION	WHAT IT TELLS YOU
First-pass acceptance	Accepted first attempts / all tasks	Early instruction quality
Accepted within budget	Accepted by <=2 repairs / all tasks	Practical completion rate
Human repair effort	Hands-on minutes per task	Hidden cost of cheap output
Escalation rate	Tasks escalated / all tasks	Model-to-task mismatch
Guardrail violations	Blocked forbidden attempts / tasks	Boundary reliability
False-green rate	Claims of PASS rejected by trusted checks	Evidence trustworthiness
Effective cost	LLM + engineer + review + retries + rework	Cost per accepted change

A tiny structured event is enough

```
task_id: payment-consumer-014
model: glm-5.3-flash
task_class: bounded-bugfix
contract_version: 2
status: ESCALATED
first_pass_accepted: false
repair_attempts: 2
failure_codes: [SEMANTIC_MISMATCH]
human_minutes: 18
validation: {targeted: pass, acceptance: fail}
next_action: replan-semantic-contract
```

COST FORMULA

Effective cost per accepted change = (subscription allocation or API charge + engineer intervention + review + escalation + subsequent rework) / accepted changes. Keep compute charges and human-time valuation as separate columns too.

04 / FEEDBACK

Extract learning from every failed run

Classify first, then decide whether to fix the plan, harness, or model.

Failure-to-improvement map

FAILURE CLASS	IMPROVE THIS LAYER	NEXT EXPERIMENT
Missing requirement	Planner contract	Add concrete behavior case
Hallucinated API	Preflight/context	Require verified signature/path
Out-of-scope edit	Enforcement layer	Harden sandbox or diff policy
Incorrect concurrency	Architecture/acceptance	Add invariant and race test; escalate
Tests green, wrong behavior	Acceptance oracle	Add independent counterexample
Repeated same repair	Routing/stop budget	Escalate earlier; isolate failure
False passing claim	Evidence mechanism	Collect machine-produced results
Correct but slow/expensive	Task sizing/routing	Compare model and granularity

A controlled comparison of Flash vs GLM 5.3

Replay the **same 20-30 representative task contracts** on identical snapshots and comparable tools. Compare three arms:

ARM A
Flash + current prompt

ARM B
Flash + stronger contract/harness

ARM C
GLM 5.3 + stronger contract/harness

HOW TO INTERPRET THE RESULT

If B beats A, your environment was under-specifying work. If C still beats B by enough to offset cost and review effort, the model has a useful capability advantage. If all arms fail the same acceptance case, inspect the plan or test oracle first.

05 / MODEL UTILITIES

Which LLM should do what?

Role boundaries first. Provider names are replaceable implementations of those roles.

Candidate role matrix from our discussion

UTILITY	CANDIDATE MODELS	BEST FIT	REQUIRED OUTPUT
Senior architect	GPT Astra; Claude Fable / Opus	High-risk design, invariants, uncertainty	Decision log + acceptance strategy
Planner / decomposer	GPT Sol; Astra; Claude Fable / Opus	Turn requirements into bounded tasks	Contracts + dependency order
General coder	Claude Sonnet; GLM 5.3; Kimi	Cross-file implementation, known interfaces	Patch + local test evidence
Lightweight coder	GLM 5.3-Flash; Claude Haiku; GPT Luna	Mechanical edits, predictable tests, scaffolding	Small patch within allowlist
Bug fixer	Flash for mechanical; GLM 5.3 / Sonnet for semantic	Reproduce, classify, repair within budget	Minimal patch + reproducible fix
Independent reviewer	Astra / Opus or second provider	Counterexamples, contracts, architecture risk	Review findings, not just praise
Trusted verifier	Tests, CI, static analysis, sandbox	Independent outcome and boundary checks	Machine-observed PASS / FAIL

SIMPLE ROUTING HEURISTIC

Low ambiguity + low blast radius: Flash or Haiku. **Moderate multi-file work:** GLM 5.3, Kimi or Sonnet. **Ambiguous correctness or risky architecture:** Astra or Opus. **After a conceptual failure:** replan before retrying.

POCKET REFERENCE

The one-page operating checklist

Print this page, place it next to your editor, and update it as your harness matures.

Before the architect hands off work

- [ ] Outcome and non-goals are explicit.
- [ ] Existing APIs and preconditions are verified (or marked for preflight).
- [ ] Architecture, failure semantics, and invariants have owners.
- [ ] Acceptance includes normal paths, counterexamples, and edge cases.
- [ ] Writable paths, protected paths, and forbidden changes are specified.
- [ ] Stop-line, retry budget, and escalation owner are explicit.

Before the coder starts editing

- [ ] Run read-only preflight; enumerate verified files and interfaces.
- [ ] Run and record baseline tests; never invent a missing API.
- [ ] Confirm agent permissions; leave protected acceptance outside its control.
- [ ] Return BLOCKED when a correctness decision is missing.

Before accepting the patch

- [ ] Trusted targeted tests and independent acceptance pass.
- [ ] Diff only touches allowed files; dependencies/interfaces are unchanged.
- [ ] Actual command outputs and remaining risks are attached.
- [ ] Repair count and failure class are recorded for the next experiment.

THE RULE WORTH REMEMBERING

The architect defines what must be true. The coder chooses how to make it true within agreed boundaries. The harness independently verifies whether it became true.

APPENDIX

Two small handoff formats to reuse tomorrow

Short enough for an agent, structured enough for your telemetry.

Read-only preflight report

```
status: READY | BLOCKED
verified_files: ["...", "..."]
verified_interfaces: ["...signature..."]
baseline: {command: "...", exit_code: 0}
expected_changed_files: ["..."]
unknowns: []
reasons_to_stop: []
```

Completion and escalation report

```
status: PASS | BLOCKED | ESCALATED
changed_files: ["..."]
verified_invariants: ["..."]
checks:
  - {command: "...", exit_code: 0, artifact: "..."}
repair_attempts: 1
failure_class: NONE | SEMANTIC | SCOPE | TEST | CONTEXT
unresolved_risks: []
next_owner: reviewer | architect | human
```

GUIDE NOTES

This is a reusable engineering framework, not a claim that any model will reliably follow every instruction. Replace example repository paths and commands. Verify current model capabilities, pricing, and provider terms when you implement routing. Organizational security policy always overrides personal workflows.

Kawaii does not mean casual about correctness. Pink bows on the outside; strong invariants on the inside. | **Ribbon Protocol - working edition 1.0**